

製造指示一覧のクエリ — 機能設計

002_DD-FN — 002_DD と 002_DD-API が使う。要件 REQ-020～REQ-027。

改版履歴

項目	値
ドキュメントID	002_DD-FN
システム名	ProductionManagementAI
サブシステム名	製造指示（Production orders）
作業項目	WI-003
作成者	Claude（ThanhTN 向け）
作成日	2026-09-22
最終更新者	Claude（ThanhTN 向け）
最終更新日	2026-09-22

版	日付	作成者	改版内容
1	2026-09-22	Claude（ThanhTN 向け）	新規作成
2	2026-09-23	Claude（ThanhTN 向け）	WI-005：UI を日本語化。引用する UI テキスト 1 か所を日本語のページング表示にした（WI-005 DEC-003）。項目・ルール・API・処理の変更はない

概要とメソッドの索引

項目	値
モジュール名	<code>ProductionOrderService.ListAsync</code> とその背後のリポジトリのメソッド（ <code>ProductionManagementAI.Application.ProductionOrders</code> 、実装は <code>ProductionManagementAI.Infrastructure.ProductionOrders</code> ）
概要	製造指示の読み取り側：検証済みの一覧のクエリを、件数付きの 1 ページの行に変える。2 つ目のサービスを作らず、001_DD-FN の既存の <code>ProductionOrderService</code> と <code>IProductionOrderRepository</code> を拡張するので、工場の時計、テレメトリー、結果の慣例は共通のまま

No	メソッド名	概要	備考
1	<code>ProductionOrderService.ListAsync</code>	製品の絞り込みを検証し、一致件数を数え、ページを取得し、納期遅れの行に印を付ける	API-PO-04
2	<code>IProductionOrderRepository.CountOrdersAsync</code>	絞り込んだ集合に対する <code>count(*)</code>	新規
3	<code>IProductionOrderRepository.ListOrdersAsync</code>	並べ替えて射影した 1 ページの行	新規
4	<code>ProductionOrderQueryExtensions.ApplyFilters</code>	設定された絞り込みだけを <code>IQueryable</code> に組み込む	新規、内部
5	<code>ProductionOrderQueryExtensions.ApplySort</code>	許可リストにある並べ替えキーを、指示番号による同順位の解決付きの順序に対応付ける	新規、内部
6	<code>ProductionOrderListMapper.ToListItem</code>	行 → <code>ProductionOrderListItem</code> 。 <code>isOverdue</code> を計算	新規、内部
7	<code>IPlantClock.Today</code>	<code>isOverdue</code> に使う工場ローカルの日付（Asia/Tokyo）	既存（001_DD-FN §6）、変更せず再利用
8	<code>IProductionOrderRepository.ProductExistsAsync</code>	製品の絞り込みの存在確認	既存（001_DD-FN）、変更せず再利用

`ListAsync` は 001_DD-FN のメソッドと同じ `Result<T>` 型を返し、`Ok` と `Invalid` のバリエーションだけを使う — 一覧のクエリが `NotFound`、`Conflict`、`RuleViolation` になることはない。想定外の例外は `Result` にせず、全体の Problem Details のハンドラーに伝える（500、MSG-E013）。

共通のユーティリティの参照

No	名前	概要	備考
1	<code>IProductionOrderRepository</code>	EF Core のデータアクセス。上の 2 つの読み取りメソッドを追加	既存のインターフェース（001_DD-FN）
2	<code>IPlantClock</code>	工場ローカルの日付と年	既存。テストの <code>FakeTimeProvider</code> で <code>isOverdue</code> が決定的になる
3	<code>ProductionOrderTelemetry</code>	<code>ProductionManagementAI.ProductionOrders</code> という名前の <code>ActivitySource</code> + <code>Meter</code>	既存。カウンター 1 つとヒストグラム 1 つを追加（可観測性を参照）
4	<code>ILogger<ProductionOrderService></code>	構造化ログ、 <code>LoggerMessage</code> のソース生成	指示番号の検索文字列を自由テキストとしてはログに記録しない — ユーザーが入力した値なので、構造化した項目としてだけ記録する
5	<code>ProductionOrderListQuery</code>	API 層が作る検証済みのクエリオブジェクト	002_DD モジュール 5 で定義。本モジュールは生のクエリ文字列を見ない

リクエストのデータ

本モジュールがデータベースに送るパラメータ。すべてバインドされたパラメータで、SQL に連結するものはない（[ai/rules/database.md](#)）。

No	名前	変数名	型	長さ	使用するメソッド	備考
1	ステータスの集合	statuses	ProductionOrderStatus[]	4 以内	ListOrdersAsync、CountOrdersAsync	status = ANY(@statuses)。空なら丸ごと省く
2	製品 ID	productId	Guid?	—	ListOrdersAsync、CountOrdersAsync、ProductExistsAsync	null なら省く
3	納期（開始）	dueFrom	DateOnly?	—	ListOrdersAsync、CountOrdersAsync	due_date >= @dueFrom
4	納期（終了）	dueTo	DateOnly?	—	ListOrdersAsync、CountOrdersAsync	due_date <= @dueTo
5	指示番号のパターン	pattern	string?	24 以内	ListOrdersAsync、CountOrdersAsync	API 層（002_DD-API）がすでに空白除去、大文字化、エスケープし、%...% で囲んでいる。EF.Functions.Like(o.OrderNumber, pattern, "\\") で使う
6	スキップ／取得件数	skip、take	int	—	ListOrdersAsync	(page - 1) × pageSize、pageSize

レスポンス／値の加工

ProductionOrderListItem（項目の一覧は 002_DD-API）は、射影した行から ProductionOrderListItemMapper.ToListItem が組み立てる：

No	名前	変数名	型	長さ	必須	値の加工	備考
1	ID	Id	Guid	—	yes	保存どおり	
2	指示番号	OrderNumber	string	13	yes	保存どおり（生成列）	
3	製品	Product	ProductSummary	—	yes	結合からの { p.Id, p.Sku, p.Name }	ProductResponse より狭いが、同じ 3 項目
4	数量	Quantity	int	—	yes	保存どおり	
5	納期	DueDate	DateOnly	—	yes	保存どおり	YYYY-MM-DD に直列化、タイムゾーン変換なし
6	ステータス	Status	string	—	yes	row.Status.ToString()	
7	納期遅れ	IsOverdue	bool	—	yes	row.DueDate < plantToday && (row.Status == Draft row.Status == InProgress)	クエリの後にそのページの行に対してメモリ上で計算する — 格納せず、SQL の述語にもしない（002_DB）

No	名前	変数名	型	長さ	必須	値の加工	備考
8	更新日時	UpdatedAt	DateTimeOffset	—	yes	updated_at_utc	UTC

`PagedResult<T>` は項目を、検証済みのクエリからそのまま返す `Total`、`Page`、`PageSize`、`Sort`、`Dir` で包む。

メソッドの設計

1. `ProductionOrderService.ListAsync`

項目	値
説明	検証済みの一覧のクエリに対して、件数付きの製造指示の 1 ページを返す
戻り値の型	<code>Task<Result<PagedResult<ProductionOrderListItem>>></code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>ProductionOrderListQuery</code>	<code>query</code>	API 層がすでに検証・正規化したもの（002_DD モジュール 5）
2	<code>CancellationToken</code>	<code>cancellationToken</code>	

戻り値

型	名前	説明
<code>Result<PagedResult<ProductionOrderListItem>></code>	<code>result</code>	ページ付きの <code>Ok</code> 、または <code>productId → MSG-E002</code> 付きの <code>Invalid</code>

処理概要：製品の絞り込みが存在するかを確認し（データベースが必要な唯一の検証）、一致件数を数え、件数が 0 でないときだけページを取得し、その後に行を対応付けて、工場の時計に対して納期遅れの行に印を付ける。件数を先に数えるので、最終ページより後のページのコストは 2 回ではなく安価なクエリ 1 回で済み、概要のための `total` も常に得られる。

処理フロー

ステップ	説明	呼び出し
1	絞り込みの形を属性に持つスパン <code>production_orders.list</code> を開始（どの絞り込みが設定されているか、並べ替えキー、方向、ページ、表示件数 — 検索文字列のテキストは含めない）	<code>ProductionOrderTelemetry.Source</code>
2	<code>query.ProductId</code> が設定されていて <code>ProductExistsAsync</code> が <code>false</code> → <code>Invalid(productId: MSG-E002)</code> を返す。結果 <code>validation_failed</code> を記録	<code>IProductionOrderRepository.ProductExistsAsync</code>
3	<code>total = await CountOrdersAsync(query)</code>	<code>\$2</code>
4	<code>total == 0</code> → ページのクエリをスキップし、空の項目のリストを使う	—
5	それ以外は <code>rows = await ListOrdersAsync(query)</code>	<code>\$3</code>
6	<code>today = plantClock.Today</code> 。各行を <code>ToListItem(row, today)</code> で対応付ける	<code>\$6</code> 、 <code>IPlantClock</code>
7	結果の件数のヒストグラムと結果 <code>success</code> を記録。操作値をそのまま返して <code>Ok(new PagedResult(...))</code> を返す	<code>ProductionOrderTelemetry</code>

最終ページより後のページはエラーではない（REQ-024）：ステップ 5 は単に行を返さず、`total` は引き続き実際の一致件数を報告するので、クライアントは「9 / 5 ページ」を正直に表示して前のページに戻る。

2. `IProductionOrderRepository.CountOrdersAsync`

項目	値
説明	クエリの絞り込みに一致する製造指示の件数
戻り値の型	<code>Task<int></code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>ProductionOrderListQuery</code>	<code>query</code>	絞り込みの項目だけを使う。並べ替えとページングは無視する
2	<code>CancellationToken</code>	<code>cancellation_token</code>	

戻り値

型	名前	説明
int	total	推定ではなく正確な件数（REQ-024）

処理概要： `db.ProductionOrders.AsNoTracking()` に §4 で絞り込みを適用し、 `CountAsync` 。 `products` との結合はない：絞り込みの述語はすべて `production_orders` にあるので、件数のクエリは 1 つのテーブルだけに触れる（002_DB）。

処理フロー

ステップ	説明	呼び出し
1	<code>db.ProductionOrders.AsNoTracking()</code> から始める	—
2	設定された絞り込みを適用	§4
3	<code>CountAsync</code>	EF Core

3. `IProductionOrderRepository.ListOrdersAsync`

項目	値
説明	並べ替えて射影した製造指示の 1 ページ
戻り値の型	<code>Task<IReadOnlyList<ProductionOrderListRow>></code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>ProductionOrderListQuery</code>	<code>query</code>	絞り込み、並べ替え、方向、ページ、表示件数
2	<code>CancellationToken</code>	<code>cancellationToken</code>	

戻り値

型	名前	説明
<code>IReadOnlyList<ProductionOrderListRow></code>	rows	要求された順序で射影した行。最大 <code>pageSize</code>

処理概要：同じ絞り込み済みの IQueryable を、表示する製品と製品の並べ替えキーのために products と結合し、\$5 で並べ替え、Skip / Take と ProductionOrderListRow への明示的な射影を行う — notes、xmin、order_year、order_seq、created_at_utc は選択しない（002_DB の読み取りの射影）。

処理フロー

ステップ	説明	呼び出し
1	db.ProductionOrders.AsNoTracking() から始める	—
2	設定された絞り込みを適用	\$4
3	product_id で db.Products と結合し、設定可能なメンバーを持つ名前付きの型 ProductionOrderJoin にする（ナビゲーションプロパティはなく、EF はメンバー初期化の射影を見通せる）	—
4	並べ替えと order_number による同順位の解決を結合したエンティティに対して適用	\$5
5	Skip((page - 1) × pageSize).Take(pageSize)	—
6	ProductionOrderListRow に射影して ToListAsync	—

ステップ 4 と 6 の順序は重要である：EF Core は、コンストラクターの射影からメンバーを読み戻す ORDER BY を翻訳できないので、並べ替えは最後の射影の後ではなく前に適用する。2 つを入れ替えると翻訳エラーで失敗する結合テストで確認している。

4. ProductionOrderQueryExtensions.ApplyFilters

項目	値
説明	クエリが実際に持つ絞り込みだけを IQueryable<ProductionOrder> に組み込む
戻り値の型	IQueryable<ProductionOrder>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	IQueryable<ProductionOrder>	source	
2	ProductionOrderListQuery	query	

戻り値

型	名前	説明
<code>IQueryable<ProductionOrder></code>	<code>filtered</code>	0～5 個の述語を組み込んだ同じソース

処理概要：各絞り込みは `if` の中で追加するので、設定されていない絞り込みは SQL をまったく加えない。これはクエリプランにとって重要である — `(@p IS NULL OR col = @p)` の述語は 002_DB が頼るインデックスを無効にする — そして、これが件数とページの両方のクエリで使う 1 つの共通メソッドになっている理由で、2 つがずれることはない。

処理フロー

ステップ	説明	呼び出し
1	<code>if (query.Statuses.Count > 0) → Where(o => query.Statuses.Contains(o.Status))</code> (<code>status = ANY(@statuses)</code> に翻訳)	—
2	<code>if (query.ProductId is Guid id) → Where(o => o.ProductId == id)</code>	—
3	<code>if (query.DueFrom is DateOnly from) → Where(o => o.DueDate >= from)</code>	—
4	<code>if (query.DueTo is DateOnly to) → Where(o => o.DueDate <= to)</code>	—
5	<code>if (query.OrderNumberPattern is string pattern) → Where(o => EF.Functions.Like(o.OrderNumber, pattern, "\\\"))</code>	<code>pg_trgm</code> のインデックス (DEC-010)

5. `ProductionOrderQueryExtensions.ApplySort`

項目	値
説明	許可リストにある並べ替えキーと方向を順序に翻訳し、常に指示番号による同順位の解決で終える
戻り値の型	<code>IOrderedQueryable<...></code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>IQueryable<ProductionOrderJoin></code>	<code>source</code>	一覧の射影の前の、 <code>products</code> と結合した絞り込み済みのクエリ
2	<code>ProductionOrderSort</code>	<code>sort</code>	文字列ではなく列挙型 — API 層がすでに許可リスト外のものを拒否している
3	<code>SortDirection</code>	<code>dir</code>	<code>Asc</code> または <code>Desc</code>

戻り値

型	名前	説明
IOrderedQueryable<...>	ordered	決定的な全順序

処理概要：列挙型に対する switch がキーの式を選び、方向が OrderBy / OrderByDescending を選ぶ。キーがすでに指示番号でない限り、常に ThenBy(o => o.OrderNumber) を付け加える。入力は列挙型だけなので、クライアントの文字列が順序に届くことはない — 許可リストはこの層での文字列比較ではなく、型システムで強制される。

処理フロー

ステップ	説明	呼び出し
1	DueDate → j.Order.DueDate （既定）	—
2	OrderNumber → j.Order.OrderNumber 。同順位の解決は不要（一意）	—
3	Product → j.Product.Sku 、次に j.Product.Name	—
4	Quantity → j.Order.Quantity	—
5	Status → 格納した文字列ではなく、ワークフローの順位の式 Draft=1, InProgress=2, Completed=3, Cancelled=4 （002_DB の並べ替えキーの対応）	—
6	UpdatedAt → j.Order.UpdatedAtUtc	—
7	昇順の ThenBy(j => j.Order.OrderNumber) を付け加える（ステップ 1、3～6）	—

方向は選んだキーにだけ適用する。同順位の解決は昇順のままなので、降順のページも安定した全順序になる。

6. ProductionOrderListMapper.ToListItem

項目	値
説明	射影した行を API の一覧の項目にし、納期遅れのフラグを決める
戻り値の型	ProductionOrderListItem
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	ProductionOrderListRow	row	

No	型	名前	説明
2	DateOnly	plantToday	行ごとではなくリクエストごとに一度読むので、ページのすべての行が「今日」を同じに判定する

戻り値

型	名前	説明
ProductionOrderListItem	item	上の「レスポンス／値の加工」を参照

処理概要：単純関数 — 同じ行と同じ日付からは常に同じ項目が得られ、これにより納期遅れのルールをデータベースなしで単体テストできる（002_BD M-08、FN-013）。

処理フロー

ステップ	説明	呼び出し
1	Id、OrderNumber、Quantity、DueDate、Status、UpdatedAt をコピー	—
2	行の 3 つの製品の列から Product を組み立てる	—
3	IsOverdue = row.DueDate < plantToday && row.Status is Draft or InProgress	—

可観測性

ai/rules/backend.md に従い、2 つ目を追加せず、既存の ProductionManagementAI.ProductionOrders の ActivitySource / Meter（001_DD-FN）を拡張する。

シグナル	名前	種類	属性／バケット
スパン	production_orders.list	Activity	filter.status_count、filter.has_product、filter.has_due_range、filter.has_order_number（真偽値／件数のみ — 検索文字列のテキストは含めない）、sort、dir、page、page_size、result.total
メトリクス	pmai.production_orders.listed	カウンター	outcome = success validation_failed error。001_DD-FN の結果の名前を再利用
メトリクス	pmai.production_orders.list_result_size	ヒストグラム	ページで返した行数 — ユーザーが実際にページを移動するか、20 を超える表示件数が使われるかを示す

ログ：拒否したクエリごとに、違反したパラメータ名とそのメッセージ ID を持つ Information の記録を 1 件（構造化した項目。補間した文字列にはしない）、想定外の失敗では例外付きの Error。指示番号の検索文字列はユーザーが入力した値で、構造化した項目としてだけ記録し、メッセージに連結しない。

未解決の決定

なし。ここでの動作は DEC-002（ページングと並べ替え）、DEC-005（ステータスの複数選択）、DEC-010（検索文字列の照合）と 002_DB のクエリ設計に従う。工場の時計と `Result<T>` / テレメトリーの慣例は 001_DD-FN から変更せず引き継ぐ。